Probability and Monte Carlo Simulation

BUS 104: Decision Analysis and Management Science (3-Lecture Module)

Danko Turcic

UC RIVERSIDE | UNIVERSITY OF CALIFORNIA | School of Business
A. GARY ANDERSON GRADUATE SCHOOL OF MANAGEMENT

Master Roadmap: Probability & Simulation in 3 Lectures

Lecture 1: Probability & Big 3

- Why business averages lie (**Flaw of Averages**).
- Random variables, PDFs, CDFs, $\mathbb{E}[X]$ & risk σ.
- **Uniform** $U(a, b)$: Flat intervals & $U(0, 1)$ engine.
- **Exponential** $\text{Exp}(\lambda)$: Waiting times & memorylessness.
- **Normal** $N(\mu, \sigma^2)$: Bell curves, Z-scores & safety stock.

Lecture 2: PRNGs & Gemini Chat

- How deterministic computers roll dice (**Clock-Spinner PRNG**).
- Why **Random Seeds** unlock reproducibility & fair policy tests.
- Transforming $U(0, 1)$ into any distribution.
- **Direct Gemini Chat Simulation:** 3-Round Venture Gamble & Retail Hoodie Inventory.

Lecture 3: Python, Queues & CIs

- Prompting Gemini to write vectorized **Python in Colab**.
- **Dual-Uncertainty Newsvendor:** Random price & demand.
- **Service Queueing ($M/M/1$):** The utilization hockey stick.
- **Confidence Intervals:** $\bar{X} \pm z\frac{s}{\sqrt{n}}$ & sample size n^*.

Lecture 1: Input Distributions $U(a, b)$, **Exp**(λ), $N(\mu, \sigma^2)$ → **Lecture 2: PRNG Engine & Direct Gemini Simulation** → **Lecture 3: Python at Scale, Queueing & 95% CIs**

Why We Simulate: The "Flaw of Averages" in Business

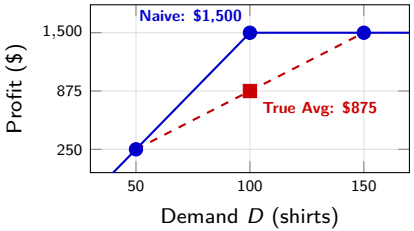
The Trap: Plugging Average Demand Into a Spreadsheet

Suppose you order $Q = 100$ event shirts ($c = \$10$, price $p = \$25$, salvage $s = \$0$). Average demand is $\mathbb{E}[D] = 100$.

- **Naive Spreadsheet (Demand $= 100$):** $\text{Profit}(\mathbb{E}[D]) = 25(100) - 10(100) = \mathbf{\$1,500}$.
- **Reality (50% Sunny $D = 150$, 50% Rainy $D = 50$):**
  - Sunny ($D = 150$): Capped at $Q = 100 \Rightarrow \$1,500$.
  - Rainy ($D = 50$): Sell $50 \Rightarrow 25(50) - 10(100) = \$250$.
- **True Expected Profit:** $\mathbb{E}[\text{Profit}(D)] = \mathbf{\$875} \neq \$1,500$!

Jensen's Inequality in Plain English

Because stockouts cap upside while leftovers punish downside: $\mathbb{E}\big[\text{Profit}(D)\big] \neq \text{Profit}\big(\mathbb{E}[D]\big)$.

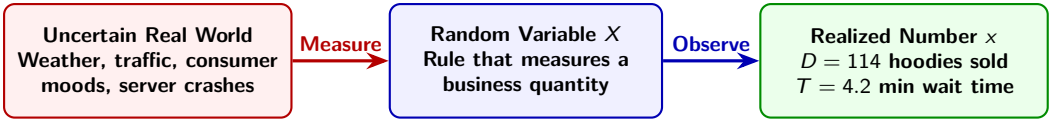


Monte Carlo Simulation samples thousands of random demands D_i to compute true average profit and risk!

What Is a Random Variable (X)?

Intuitive Definition (Zero Measure-Theory Jargon)

A **Random Variable** X is simply a **numerical thermometer** that attaches a concrete number to an uncertain future business outcome before it happens.



Notation Rule: Capital X vs. Lowercase x

- **Capital X (or D, T):** The uncertain variable *before* we roll the dice (e.g., tomorrow's demand D).
- **Lowercase x:** One specific numerical realization *after* rolling the dice (e.g., $x = 114$ units).

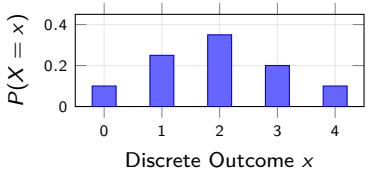
Why Simulation Needs Random Variables

Every uncertain input in a spreadsheet or Python model (daily demand, supplier lead time, clearing price) is modeled as a random variable drawn from a **probability distribution**.

Two Flavors of Uncertainty: Discrete vs. Continuous

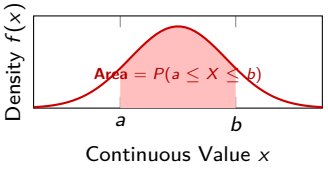
1. Discrete Random Variables (Counting)

- Takes distinct, countable values $(0, 1, 2, \dots)$.
- **Examples:** Defective chips in a box, ships at a port, coin flips.
- **PMF:** Bar height $p(x) = P(X = x)$, with $\sum_x p(x) = 1$.



2. Continuous Random Variables (Measuring)

- Can take *any* decimal value on a continuum (4.382 min).
- **PDF** $f(x)$**:** Probability is the **shaded area** over an interval ($P(X = a) = 0$ at a single point):
  $P(a \le X \le b) = \int_a^b f(x)\,dx$.



The Cumulative Distribution Function (CDF): $F(x) = P(X \leq x)$

What Does the CDF $F(x)$ Measure?

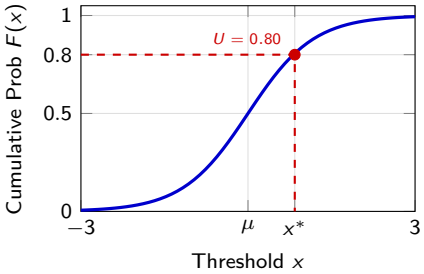
The CDF accumulates all probability up to threshold x:

$$F(x) = P(X \leq x) \quad \text{(always rises from 0 to 1)}$$

Three Ways Managers Use the CDF

1. **In-Stock Service Level:** If demand is D and we stock Q units, the probability we meet all demand without stocking out is $F(Q) = P(D \leq Q)$.
2. **Tail Risk:** $P(X > x) = 1 - F(x)$.
3. **Interval Probability:**

$$P(a \leq X \leq b) = F(b) - F(a)$$



Preview of Lecture 2 (Inverse CDF)

Running the CDF curve **backward** from height $U \in (0, 1)$ on the vertical axis to x^* on the horizontal axis is how computers simulate any distribution!

Summarizing a Distribution: Expected Value $\mathbb{E}[X]$ vs. Risk σ

1. Expected Value $\mu = \mathbb{E}[X]$ (Center of Gravity)

- Probability-weighted average of all outcomes:

$$\mathbb{E}[X] = \sum_x x\, P(X = x) \quad \text{or} \quad \int_{-\infty}^{\infty} x\, f(x)\, dx$$

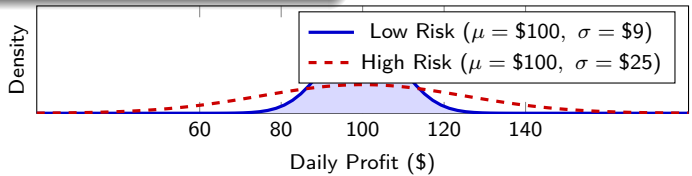
- **Simulation Meaning:** If you simulate X for $100,000$ independent days, the sample average $\bar{X}$ converges to $\mathbb{E}[X]$.

2. Standard Deviation σ (Spread / Volatility)

- Measures typical distance from the mean μ:

$$\text{Var}(X) = \sigma^2 = \mathbb{E}\left[(X - \mu)^2\right], \quad \sigma = \sqrt{\text{Var}(X)}$$

- **Why σ Beats σ^2:** Variance σ^2 is in *squared units* (dollars^2); σ is back in original business units (dollars, minutes, units).



Distribution #1: The Uniform Distribution (Maximum Ignorance)

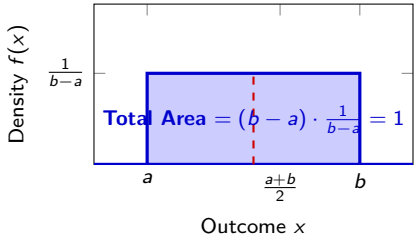
When to Use the Uniform Distribution

Use the Uniform distribution when you only know the **minimum** a and **maximum** b, and every value in between is **equally likely**.

Discrete vs. Continuous Uniform

- **Discrete Uniform** $\{1, 2, \ldots, k\}$**:** Rolling a fair die or picking a random customer ID; each integer has probability $1/k$.
- **Continuous Uniform** $X \sim U(a, b)$**:** Any decimal inside $[a, b]$ is equally dense. Because total rectangle area must equal 1:

$$(b - a) \cdot f(x) = 1 \implies f(x) = \frac{1}{b - a}$$



Business

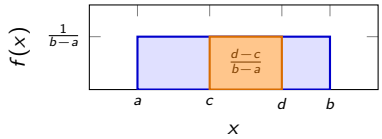
Examples: Unscheduled arrival within a 2-hour delivery window; first-pass bid range $[a, b]$.

Continuous Uniform $U(a, b)$: Length Ratios, Mean & Spread

1. Probability = Simple Ratio of Interval Lengths

For any sub-interval $[c, d]$ inside $[a, b]$, probability is just the sub-interval width divided by total width:

$$P(c \le X \le d) = \frac{d-c}{b-a}, \qquad F(x) = \frac{x-a}{b-a}$$



Why Divide by $\sqrt{12} \approx 3.464$?

Because a flat rectangle pushes probability into both edges $[a, b]$, its spread σ is roughly 29% of the total range $(b - a)$.

2. Mean (Midpoint) and Standard Deviation

- **Expected Value (Exact Midpoint):** $\mathbb{E}[X] = \dfrac{a+b}{2}$.
- **Variance & Standard Deviation:**

$$\mathrm{Var}(X) = \frac{(b-a)^2}{12}, \qquad \sigma = \frac{b-a}{\sqrt{12}} \approx 0.2887\,(b-a)$$

Worked Example: Uniform Freight Delivery Window

Scenario: Morning Warehouse Receiving Dock

A supplier truck is promised between **8:00 AM and 10:00 AM**. Let $X \sim U(0, 120)$ be the arrival time in minutes after 8:00 AM ($a = 0$, $b = 120$).

Step-by-Step Calculations

1. **Expected Arrival Time:**
   $\mathbb{E}[X] = \dfrac{0 + 120}{2} = \mathbf{60}$ min (9:00 AM).
2. **Standard Deviation of Arrival:**
   $$\sigma = \frac{120 - 0}{\sqrt{12}} = \frac{120}{3.4641} = \mathbf{34.64} \text{ min}$$
3. **Prob. Truck Arrives Between 8:30 and 9:15 AM ($30 \leq X \leq 75$):**
   $$P(30 \leq X \leq 75) = \frac{75 - 30}{120 - 0} = \frac{45}{120} = \mathbf{0.375}\ (\mathbf{37.5\%})$$

Dock Staffing Question

If the dock crew takes a coffee break from **9:40 AM to 10:00 AM** ($100 \leq X \leq 120$), what is the risk the truck arrives during the break?

$$P(X \geq 100) = \frac{120 - 100}{120} = \frac{20}{120} = \mathbf{16.7\%}$$

Takeaway: With a uniform window, every 20-minute block carries the exact same 16.7% arrival risk!

Why Standard Uniform $U(0,1)$ Is the Heartbeat of Simulation

Stretching a $U(0,1)$ Decimal Into Any Interval $[a,b]$

Every spreadsheet ('=RAND()') and programming language starts by generating a standard uniform random decimal $U \sim U(0,1)$. To simulate $X \sim U(a,b)$, simply shift and stretch U:

$$\boxed{X = a + (b-a) \cdot U}$$

- When $U = 0 \implies X = a$ (minimum). When $U = 0.5 \implies X = \frac{a+b}{2}$ (midpoint). When $U = 1 \implies X = b$ (maximum).

In Google Sheets / Excel

To simulate a delivery time $X \sim U(0,120)$ or price $P \sim U(9,11)$:

- `=0 + (120 - 0) * RAND()`
- `=9 + (11 - 9) * RAND()`

In Python (`numpy.random`)

```
import numpy as np
# Draw 5 random arrivals from U(0, 120)
u = np.random.uniform(0, 1, size=5)
x = 0 + (120 - 0) * u
# Or directly: np.random.uniform(0, 120, size=5)
```

Interactive Exercise 1: Flash-Sale Cloud Server Load

Scenario: Ticketing Website Drop

During a 1-minute concert ticket drop, peak server traffic X (in thousands of requests/min) is modeled as continuously uniform between $a = 40$ and $b = 100$, so $X \sim U(40, 100)$. Your cloud auto-scaler triggers emergency backup servers whenever load exceeds 85 **thousand requests/min**.

Your Tasks (3 Minutes)

1. Compute the **expected server load** $\mathbb{E}[X]$ and **standard deviation** σ.
2. What is the probability $P(X > 85)$ that emergency backup servers are triggered?
3. Suppose your random number generator draws $U = 0.75$. What simulated server load X does that produce?

FORMULA & PARAMETER REFERENCE: $X \sim U(a, b)$: $\mathbb{E}[X] = \dfrac{a+b}{2}$, $\sigma = \dfrac{b-a}{\sqrt{12}}$ ($\sqrt{12} \approx 3.464$),

$P(c \le X \le d) = \dfrac{d-c}{b-a}$, $X = a + (b-a)U$.

Distribution #2: The Exponential Distribution Exp(λ)

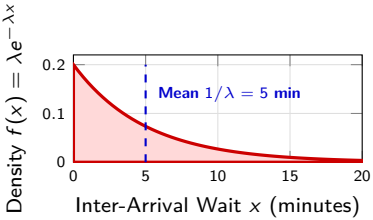
What Does Exp(λ) Model?

Models the **waiting time (inter-arrival gap)** $X \geq 0$ between independent random events occurring at average rate λ:

- Time between customer arrivals at a coffee bar or EV charger.
- Time between helpdesk tickets or machine breakdowns.

Crucial Parameter Rule: Rate λ vs. Mean Wait $1/\lambda$

- **Rate λ:** Avg. arrivals per unit time ($\lambda = 0.2$ arrivals/min).
- **Expected Wait & Std. Dev. (Identical!):** $\mathbb{E}[X] = 1/\lambda$ and $\sigma = 1/\lambda$.
- *Match units!*
  $\lambda = 12/\text{hr} \iff \lambda = 0.20/\text{min} \iff \mathbb{E}[X] = 5$ min.



Notice the curve peaks at $x = 0$: short gaps (bursts of arrivals) are very common, alongside occasional long droughts!

Exponential Tail Probabilities & Intuitive Memorylessness

Simple Exponential CDF & Survival Formulas

For waiting time $X \sim \text{Exp}(\lambda)$ $(x \geq 0)$:

- **Prob. arrival occurs within x minutes (CDF):**

$$F(x) = P(X \leq x) = 1 - e^{-\lambda x}$$

- **Prob. you wait longer than x minutes (Tail):**

$$P(X > x) = e^{-\lambda x}$$

Why Queues Form: Bursts + Droughts

Because $f(x)$ is highest near 0, random arrivals naturally **clump together in bursts** (overwhelming servers) followed by **long idle droughts** (starving servers).

The Memoryless Property (No Algebra Needed!)

$$P(X > t + s \mid X > t) = P(X > s) = e^{-\lambda s}$$

What this means in plain English:

- The process has **zero memory** of how long you have already waited.
- If average time between customers is 5 minutes and the shop has been empty for 10 fruitless minutes, the next customer is **NOT "due"**!
- Your remaining wait from right now still averages a fresh 5 minutes, just like flipping a coin after 5 tails.

Worked Example: Campus Coffee Bar Arrivals

Scenario: Morning Espresso Line

Customers arrive randomly at an average rate of $\lambda = 12$ **customers per hour**.

- **Step 0 (Convert to Minutes):** $\lambda = \frac{12}{60} = \mathbf{0.20}$ cust/min $\implies$ Mean Wait $\mathbb{E}[X] = \sigma = \frac{1}{0.20} = \mathbf{5}$ min.

1. Rush Clumping vs. Long Droughts

- **Burst ($X \leq 2$ min):** Prob. the next customer arrives within 2 minutes:

$$P(X \leq 2) = 1 - e^{-0.20(2)} = 1 - e^{-0.4} = \mathbf{33.0\%}$$

- **Drought ($X > 10$ min):** Prob. barista waits over 10 minutes:

$$P(X > 10) = e^{-0.20(10)} = e^{-2} = \mathbf{13.5\%}$$

2. Memoryless Check

Suppose 8 minutes have already passed with zero customers. What is the probability the barista waits **at least 10 more minutes** $(X > 18 \mid X > 8)$?

$$P(X > 18 \mid X > 8) = P(X > 10) = e^{-2} = \mathbf{13.5\%}$$

The clock resets completely at minute 8!

Interactive Exercise 2: EV Fast-Charging Station Arrivals

Scenario: Highway EV Supercharger Plaza

Electric vehicles arrive at a highway charging plaza randomly at an average rate of $\lambda = 3$ **cars per hour**. Let X be the inter-arrival time in **minutes** between consecutive EVs.

Your Tasks (3 Minutes)

1. Convert the arrival rate to **cars per minute** (λ_{min}), and find the **expected wait** $\mathbb{E}[X]$ and **standard deviation** σ in minutes.
2. What is the probability $P(X > 30)$ that more than 30 minutes pass before the next EV arrives?
3. Suppose 20 minutes have already elapsed with no arrival. What is the probability that **at least 30 additional minutes** pass before the next EV arrives ($P(X > 50 \mid X > 20)$)?

FORMULA & PARAMETER REFERENCE: $X \sim \text{Exp}(\lambda)$: $\mathbb{E}[X] = \sigma = \frac{1}{\lambda}$, $P(X \leq x) = 1 - e^{-\lambda x}$, $P(X > x) = e^{-\lambda x}$ ($e^{-1.5} \approx 0.2231$), Memoryless: $P(X > t + s \mid X > t) = P(X > s)$.

Distribution #3: The Normal Distribution $N(\mu, \sigma^2)$

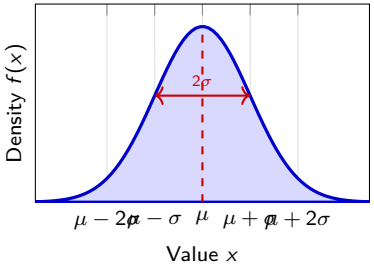
Why the Bell Curve Appears Everywhere

By the **Central Limit Theorem (CLT)**, whenever a business metric is the sum of many small, independent influences (hundreds of individual shoppers, daily market trades, manufacturing tolerances), its distribution converges to a symmetric **Normal Bell Curve**:

$$X \sim N(\mu, \sigma^2)$$

Two Parameters Control the Entire Curve

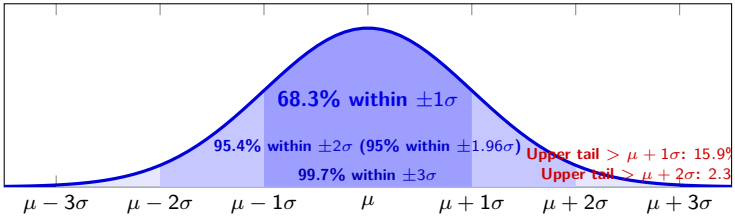
- **Mean μ:** Shifts the center peak left or right (Mean = Median = Mode).
- **Standard Deviation σ:** Controls the width (distance from peak μ to the inflection points of the bell).



50/50

Symmetry: Exactly 50% of probability lies below μ and 50% lies above μ.

The 68–95–99.7 Empirical Rule & One-Sided Tail Risks



Symmetric Central Intervals

- $\mu \pm 1\sigma$ captures **68.3%** of outcomes (leaves 31.7% in both tails combined).
- $\mu \pm 2\sigma$ captures **95.4%** (exact 95% is $\mu \pm 1.96\sigma$).

One-Sided Service Level Benchmarks

Because the bell curve is symmetric, the leftover tail splits evenly in half:

- $P(X \leq \mu + 1\sigma) = 50\% + 34.1\% = \mathbf{84.1}\%$ (15.9% stockout).
- $P(X \leq \mu + 2\sigma) = 50\% + 47.7\% = \mathbf{97.7}\%$ (2.3% stockout).

Standardization & Z-Scores: The Universal Normal Ruler

1. Forward ($X \to Z$): Finding Probability

To find the probability of any threshold X, count how many standard deviations X sits above or below μ:

$$\boxed{Z = \frac{X - \mu}{\sigma}} \sim N(0,1)$$

Then look up $\Phi(Z) = P(N(0,1) \leq Z)$:

- Sheets: `=NORM.DIST(x, mu, sigma, TRUE)`
- Python: `scipy.stats.norm.cdf(z)`

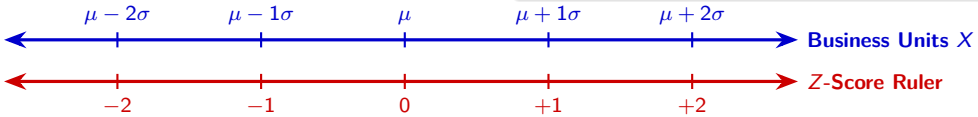
2. Backward ($Z \to X$): Target Service Levels

To find the inventory level X needed to achieve target service level α (or to simulate a random Normal draw):

$$\boxed{X = \mu + Z \cdot \sigma}$$

Key Target Z-Multipliers to Memorize:

- 84.1% service $\implies$ $Z = \mathbf{1.00}$;
  90.0% $\implies$ $Z = \mathbf{1.28}$
- 95.0% service $\implies$ $Z = \mathbf{1.645}$;
  97.5% $\implies$ $Z = \mathbf{1.96}$



Worked Example: Retail Inventory Service Level & Safety Stock

Scenario: Regional Distribution Center

Weekly demand for wireless earbuds is Normally distributed with mean $\mu = 500$ units and standard deviation $\sigma = 80$ units: $D \sim N(500, 80^2)$.

Part A: Evaluating Order Quantity $Q = 600$

If the warehouse stocks $Q = 600$ units:

1. **Compute Z-Score:**

$$Z = \frac{Q - \mu}{\sigma} = \frac{600 - 500}{80} = +\mathbf{1.25}$$

2. **In-Stock Service Level:**

$$P(D \leq 600) = \Phi(1.25) = \mathbf{89.4}\%$$

3. **Stockout Risk:** $1 - 0.894 = \mathbf{10.6}\%$.

Part B: Sizing Inventory for a 95% Service Level

How many units Q^* must we stock so we only stock out 5% of weeks ($P(D \leq Q^*) = 0.95$)?

1. For 95% one-sided service, $Z_{0.95} = \mathbf{1.645}$.
2. **Safety Stock Buffer:**

$$\text{SS} = Z \cdot \sigma = 1.645 \times 80 = \mathbf{131.6} \approx \mathbf{132} \text{ units}$$

3. **Total Order Quantity:**

$$Q^* = \mu + Z\sigma = 500 + 132 = \mathbf{632} \text{ units}$$

Interactive Exercise 3: Concert Merch Inventory & Safety Stock

Scenario: Arena Tour Hoodie Stand

Demand D for tour hoodies at an arena concert follows a Normal distribution with mean $\mu = 400$ hoodies and standard deviation $\sigma = 50$ hoodies:

$$D \sim N(400, 50^2)$$

Your Tasks (3 Minutes)

1. Suppose the merch manager brings $Q = 450$ hoodies. Compute the Z-score and find the **in-stock service level** $P(D \leq 450)$ and **stockout probability** $P(D > 450)$.
2. What is the probability that demand lands between 350 and 450 hoodies ($P(350 \leq D \leq 450)$)?
3. How many hoodies Q^* must the manager bring to achieve a 97.5% **in-stock service level** (2.5% stockout risk)?

FORMULA & PARAMETER REFERENCE: $Z = \dfrac{X - \mu}{\sigma}$, $X = \mu + Z\sigma$, $\Phi(1.0) = 0.8413$,
$P(\mu - 1\sigma \leq X \leq \mu + 1\sigma) = 0.6827$, $\Phi(1.96) = 0.975$.

Empirical Distributions: Simulating Directly from Historical Data

What If Data Doesn't Fit a Smooth Textbook Curve?

Real POS-scanner or call-center logs often have multi-modal spikes or weird gaps. Instead of forcing a Normal curve, we simulate directly from the **Empirical Distribution**:

Demand d_k	Days	Rel. Freq p_k	Cum. $F(d_k)$
10 units	20	0.20	0.20
20 units	50	0.50	0.70
30 units	30	0.30	1.00

```
import numpy as np
# Resample 10,000 days from historical probabilities:
d = np.random.choice([10, 20, 30], size=10000,
                     p=[0.20, 0.50, 0.30])
```

TO SIMULATE	USE BUILT-IN EXCEL FORMULA
Random number	=RAND()
Continuous uniform distribution Between a and b	$=a+(b-a)$*RAND()
Discrete uniform distribution Between a and b	=INT($a+(b-a+1)$*RAND()) or =RANDBETWEEN(a,b)
Normal distribution Mean = μ; Standard deviation = σ	=NORMINV(RAND(),μ,σ)
Exponential distribution Mean rate = μ	$=-(1/\mu)$*LN(RAND())
Discrete general distribution **Two outcomes only:** A and B Probability of outcome $A = p$	=IF(RAND()<p, A, B)
Discrete general distribution **More than two outcomes:** *Range1* = Cell range containing lower limits of the random number intervals *Range2* = Cell range containing the variable values	=LOOKUP(RAND(), *Range1*, *Range2*)

Bootstrapping Historical Logs

In Python, `np.random.choice(history, size=10000, replace=True)` resamples past days with replacement—preserving every real-world spike!

Master Cheat Sheet: Choosing the Right Distribution

Distribution	Shape & Support	When to Use in Business	Mean & Std.	Python (`np.random`)
Uniform $U(a, b)$	Flat rectangle on $[a, b]$	Only min a & max b known; delivery windows; PRNG $U(0, 1)$	$\mathbb{E}[X] = \frac{a+b}{2}$ $\sigma = \frac{b-a}{\sqrt{12}}$	`uniform(a, b, n)`
Exponential Exp(λ)	Right-skewed decay on $[0, \infty)$	Inter-arrival gaps between random customers, breakdowns	$\mathbb{E}[X] = \frac{1}{\lambda}$ $\sigma = \frac{1}{\lambda}$	`exponential( scale=1/lam, size=n)`
Normal $N(\mu, \sigma^2)$	Symmetric bell curve on $(-\infty, \infty)$	Aggregate retail demand, forecast errors, CLT means	$\mathbb{E}[X] = \mu$ $\sigma = \sigma$	`normal(mu, sig, n)`
Empirical / Discrete	Custom bars at observed $\{x_k\}$	Raw POS history, irregular outcomes, multi-modal data	$\mathbb{E}[X] = \sum x_k p_k$	`choice(vals, p=probs, size=n)`

Watch Out for the NumPy Exponential Trap!

In Python's `np.random.exponential(scale=...)`, `scale` is the **mean wait** $1/\lambda$, NOT the rate λ! Always pass `scale = 1/lam`.

Lecture 1 Wrap-Up & Bridge to Lecture 2

What We Mastered in Lecture 1

- Why plugging $\mathbb{E}[D]$ into an asymmetric profit formula fails (**Flaw of Averages**).
- How PDFs (area) and CDFs ($P(X \leq x)$) quantify uncertainty.
- How to recognize and calculate with the **Big Three**:
  1. **Uniform** $U(a, b)$**:** Length ratios & $X = a + (b - a)U$.
  2. **Exponential Exp**(λ)**:** Rate vs. mean wait $1/\lambda$, $e^{-\lambda x}$ tails, and memorylessness.
  3. **Normal** $N(\mu, \sigma^2)$**:** Z-scores and safety stock $Q^* = \mu + Z\sigma$.

Coming Up in Lecture 2

- **Inside the Black Box:** How can a strictly deterministic computer generate "random" numbers $U \sim U(0, 1)$?
- **The Clock-Arithmetic Spinner** and why **Random Seeds** let managers run fair apples-to-apples policy tests.
- **Inverse CDF Transform:** Turning $U \sim U(0, 1)$ into Exponential and Normal draws.
- **Hands-On Gemini Simulation:** Prompting Gemini Chat to simulate multi-round gambles and retail inventory decisions directly!

Lecture 2: PRNGs & Running Simulations Directly in Gemini

Part 1: How Computers Roll Dice (Slides 26–31)

- **The Computer Paradox:** Physical randomness vs. Pseudo-Random Number Generators (PRNGs).
- **The Clock-Arithmetic Spinner:** Intuitive view of $X_{n+1} = (aX_n + c) \bmod m$.
- **Why Random Seeds Matter:** Reproducibility & Common Random Numbers for policy testing.
- **Inverse CDF & Bucket Lookup:** Turning $U \sim U(0, 1)$ into any distribution.

Part 2: Direct Gemini Simulation (Slides 32–45)

- **Two Gemini Modes:** Direct Chat Simulation vs. Python Code Generation.
- **The 5-Part Gemini Prompt Template**.
- **Case 1 (Multi-Round Gamble):** Why $\mathbb{E}[W_3] = \$8$ hides a 50% bankruptcy rate.
- **Case 2 (UCR Homecoming Hoodies):** Comparing order quantities $Q = 100$ vs. $Q = 120$ in Gemini.
- **AI Literacy:** Auditing LLM simulations & spotting flawed prompts.

The Computer Paradox: How Can a Deterministic Machine Roll Dice?

1. True Physical Randomness (TRNG)

- Generated by physical chaos: atmospheric radio noise, radioactive decay, or thermal silicon jitter.
- **Pros:** Truly unpredictable (great for cryptography).
- **Cons for Business Analytics:** Slow, requires hardware sensors, and **impossible to replay** when debugging a spreadsheet or comparing two inventory policies!

2. Pseudo-Random Number Generators (PRNG)

- A **100% deterministic mathematical recipe** that takes a starting number (**Seed** X_0) and scrambles it into a sequence $U_1, U_2, \cdots \in [0, 1)$ that *passes statistical tests for randomness*.
- **Used by:** Excel `RAND()`, Python `numpy.random`, and Gemini's Python engine.
- **Key Feature:** Same seed $\Longrightarrow$ **exact same sequence** every time!



Inside a PRNG: The "Clock-Arithmetic Spinner" Intuition

How a Linear Congruential Generator (LCG) Works

Imagine a giant clock with m hours $(0, 1, 2, \ldots, m-1)$. Starting from seed X_0, each step does two things:

1. **Multiply, Shift & Wrap Around the Clock (mod m):** $X_{n+1} = (a \cdot X_n + c) \bmod m$.
   ($\bmod m$ *means "divide by m and keep the clock remainder"—just like* 15 *o'clock on a* 12*-hour clock wraps to* 3 *o'clock!*)
2. **Divide by m to Squash Into $[0, 1)$:** $U_{n+1} = \dfrac{X_{n+1}}{m} \in [0, 1)$.

Real-World Scale

Modern PRNGs use huge clocks ($m = 2^{32} \approx 4.3$ billion, or Mersenne Twister / PCG64) so the spinner never repeats in your lifetime!



16-Hour Toy Clock ($m = 16$)
Multiply by a, add c, wrap around!

Toy Walkthrough: Watching a 16-Hour Clock Generate $U(0,1)$

Toy Clock Settings: Clock Size $m = 16$, Multiplier $a = 5$, Shift $c = 1$, Seed $X_0 = 3$

Formula: $X_{n+1} = (5X_n + 1) \bmod 16$, and pseudo-random decimal $U_{n+1} = X_{n+1}/16$:

Step n	**Prev.** X_{n-1}	**Spin:** $5X_{n-1}+1$	**Clock Remainder** X_n **(mod 16)**	**Uniform Draw** $U_n = X_n/16$
Seed 0	—	—	$X_0 = \mathbf{3}$	—
Step 1	3	$5(3)+1 = 16$	$16 \bmod 16 = \mathbf{0}$	$0/16 = \mathbf{0.0000}$
Step 2	0	$5(0)+1 = 1$	$1 \bmod 16 = \mathbf{1}$	$1/16 = \mathbf{0.0625}$
Step 3	1	$5(1)+1 = 6$	$6 \bmod 16 = \mathbf{6}$	$6/16 = \mathbf{0.3750}$
Step 4	6	$5(6)+1 = 31$	$31 \bmod 16 = \mathbf{15}$	$15/16 = \mathbf{0.9375}$
Step 5	15	$5(15)+1 = 76$	$76 \bmod 16 = \mathbf{12}$	$12/16 = \mathbf{0.7500}$

Two Instant Insights from This Table

1. **Looks Jumpy, Yet 100% Deterministic:** The decimals $0.0000 \to 0.0625 \to 0.3750 \to 0.9375 \to 0.7500$ bounce all over $[0,1)$, yet anyone starting at Seed $X_0 = 3$ gets the *exact same numbers*.
2. **You Never Compute This by Hand Again:** Python's `np.random.seed(42)` handles the clock arithmetic automatically!

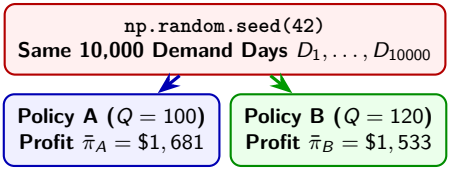
Why Random Seeds Are a Business Analytics Superpower

Superpower 1: Reproducibility & Auditing

- If your CFO asks *"How did you get* $1,681 *expected profit on Slide 12?"*, setting `np.random.seed(42)` lets any colleague re-run your script and reproduce the **exact same dollar figure to the penny**.

Superpower 2: Common Random Numbers (CRN)

- Suppose we compare **Order Policy A** ($Q = 100$) vs. **Order Policy B** ($Q = 120$).
- If we use *different* random demands for A and B, Policy A might just get lucky with sunny weather!
- Fixing the seed tests both policies on the **exact same 10,000 simulated days** (apples-to-apples!).




```
import numpy as np
np.random.seed(42) # Lock the random stream!
demands = np.random.normal(100, 20, size=10000)
# Test BOTH policies on the exact same demands:
profit_100 = np.mean(np.minimum(demands, 100)*50 -
     100*30)
profit_120 = np.mean(np.minimum(demands, 120)*50 -
     120*30)
```

From $U(0,1)$ to Any Distribution: The Inverse CDF Method

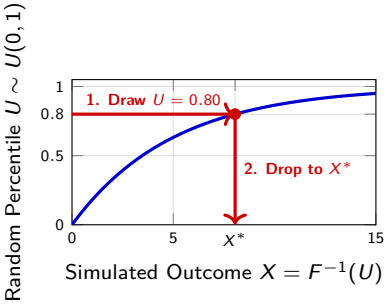
How Do We Turn $U \sim U(0,1)$ Into Exponential or Normal?

Since every CDF $F(x) = P(X \le x)$ climbs from 0 to 1:

1. Draw a uniform random height $U \sim U(0,1)$ on the vertical axis.
2. Read horizontally across to the CDF curve and drop down to find $X = F^{-1}(U)$!

The Big Three Inverse Formulas

- **Uniform** $U(a,b)$: $X = a + (b-a)U$
- **Exponential Exp(λ)**: Solve $1 - e^{-\lambda X} = U \implies \mathbf{X = -\frac{1}{\lambda}\ln(1-U)}$.
- **Normal** $N(\mu, \sigma^2)$: $X = \mu + \sigma\,\Phi^{-1}(U)$ (Sheets: `=NORM.INV(RAND(), mu, sigma)`).

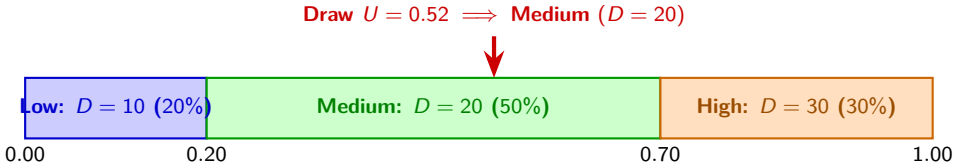


Example ($\lambda = 0.2$): If $U = 0.80$, then $X^* = -\frac{1}{0.2}\ln(1-0.80) = -5\ln(0.20) = \mathbf{8.05}$ min.

Discrete Bucket Lookup: Simulating Coins & Empirical Tables

Carving the $[0, 1)$ Number Line Into Probability Buckets

For a discrete random variable, we stack the probabilities p_k along the $[0, 1)$ ruler into cumulative buckets $[F_{k-1}, F_k)$ and check which bucket $U \sim U(0, 1)$ lands in!



Fair Coin Flip ($p = 0.50$)

- If $0 \leq U < 0.50 \implies$ **Heads (Win)**.
- If $0.50 \leq U < 1.00 \implies$ **Tails (Lose)**.

3-Scenario Demand Lookup

- $0.00 \leq U < 0.20 \implies D = 10$
- $0.20 \leq U < 0.70 \implies D = 20$
- $0.70 \leq U < 1.00 \implies D = 30$

Two Ways to Use Google Gemini for Monte Carlo Simulation

Mode 1: Direct Simulation in Gemini Chat (Today)

You paste a plain-English prompt into Gemini and ask Gemini to run the simulation and report the results directly in the chat window.

- **Best for:** Rapid prototyping, sanity checks, exploring small-to-medium models, and classroom brainstorming.
- **Under the Hood:** When you include *"Use Python in the background to run 10,000 trials"*, Gemini writes and executes real Python code on Google's servers and prints the summary table!

Mode 2: Gemini Writes Python for Colab (Lecture 3)

You prompt Gemini to generate a clean Python script, then click "Open in Colab" to run 100,000+ trials yourself.

- **Best for:** Large-scale optimization across dozens of order quantities Q, multi-server queueing systems, custom histograms, and formal Confidence Intervals.
- **100% Verifiable:** You see and control every line of code and random seed.

The 5-Part Prompt Blueprint for Direct Gemini Simulation

Never Give an LLM a Vague 1-Line Prompt! Always Specify These 5 Ingredients:

1. **1. Context & Decision Variable:** What business problem are we analyzing, and what policy choices are we comparing (e.g., $Q = 100$ vs. $Q = 120$)?
2. **2. Exact Probability Distributions & Parameters:** Specify both the family and exact parameters (e.g., $D \sim N(\mu = 100, \sigma = 20)$, NOT "average demand is 100").
3. **3. Step-by-Step Business Logic / Payoff Formula:** Spell out how revenue, cost, salvage, or sequential rounds work.
4. **4. Number of Trials & Execution Instruction:** Ask Gemini to *"Run a Monte Carlo simulation of* $N = 10,000$ *trials using Python in the background with seed 42."*
5. **5. Required Output Table:** Ask for a first-5-trials audit table PLUS summary metrics (Mean, Std. Dev., and Risk/Bankruptcy/Stockout probability).

Gemini Case 1: The Three-Round Venture Gamble

The Rules of the Game (Aggressive Fractional Betting)

You start with initial capital $W_0 = \$8$. You play **3 sequential rounds** of a fair coin flip (50% Heads = win the bet, 50% Tails = lose the bet):

- **Round 1:** You bet $\frac{1}{4}$ **of your current wealth** ($B_1 = \frac{1}{4}W_0 = \$2$).
- **Round 2:** You bet $\frac{1}{2}$ **of whatever wealth you have after Round 1** ($B_2 = \frac{1}{2}W_1$).
- **Round 3:** You go **ALL-IN**, betting 100% **of whatever wealth you have after Round 2** ($B_3 = W_2$).

Quick Example Path: `H, T, H`

- Start: $W_0 = \$8$. Bet $\$2 \rightarrow$ **H** $\implies W_1 = \$10$.
- Round 2: Bet $\frac{1}{2}(\$10) = \$5 \rightarrow$ **T** $\implies W_2 = \$5$.
- Round 3: Bet all $\$5 \rightarrow$ **H** $\implies W_3 = \mathbf{\$10}$.

Classroom Intuition Poll

1. Since every coin flip is fair (50/50), what do you think your **expected final wealth** $\mathbb{E}[W_3]$ is?
2. Would you invest your retirement savings in this 3-round strategy? Why or why not?

Interactive Exercise 4: Simulate the 3-Round Gamble in Gemini

COPY & PASTE THIS PROMPT INTO GOOGLE GEMINI:

```
Run a Monte Carlo simulation of 10,000 trials (using Python in the background with seed 42) for this 3-round coin-flip game: I start with $8. Each round is a fair 50/50 coin flip (Heads = win bet, Tails = lose bet). Round 1: bet 1/4 of current wealth. Round 2: bet 1/2 of current wealth after Round 1. Round 3: bet 100% (all) of current wealth after Round 2. Show: (1) a table of the first 5 simulated trials, (2) a table of all 8 exact possible 3-flip paths and their final payouts, and (3) the simulated average final wealth E[W3], standard deviation, and probability of ending with $0.
```

Your Tasks (4 Minutes — Pair Up with Gemini & Hand Math)

1. **By Hand:** Trace the best possible path (`H, H, H`) and worst paths (any path ending in `T` on Round 3).
2. **In Gemini:** Run the prompt above. What is the **expected final wealth** $\mathbb{E}[W_3]$, and what is the **probability of total ruin ($W_3 = \$0$)**?

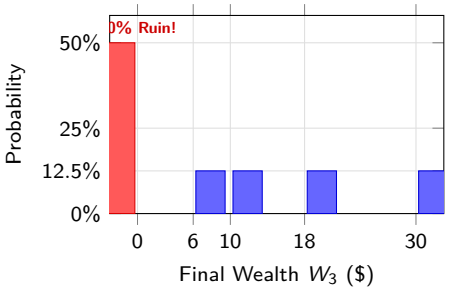
FORMULA & PARAMETER REFERENCE: $W_0 = \$8, \quad W_1 = W_0 \pm \frac{1}{4}W_0, \quad W_2 = W_1 \pm \frac{1}{2}W_1, \quad W_3 = W_2 \pm W_2.$

Each 3-flip path has probability $\left(\frac{1}{2}\right)^3 = \frac{1}{8} = 0.125$.

Managerial Takeaway: Why Expected Value Alone Can Kill a Firm

Looking Only at $\mathbb{E}[W_3] = \$8.00$ Is Blind

- A naive analyst who only checks $\mathbb{E}[W_3] = \$8.00$ says: *"This strategy is break-even on average!"*
- A simulation analyst looks at the **entire output distribution**:
  - **50.0**% chance of **total ruin** (\$0).
  - **12.5**% chance of losing money (\$6).
  - **37.5**% chance of making money (\$10, \$18, \$30).



The simulated wealth histogram is heavily skewed: half of all startups die at \$0, funded by a 12.5% jackpot at \$30.

The Core Lesson of Simulation

Simulation doesn't just give you a single average $\bar{X}$—it reveals **tail risk, skewness, and probability of ruin** before you bet the company!

Gemini Case 2: UCR Homecoming Hoodie Inventory (Newsvendor)

Business Scenario: One-Shot Perishable Merch Order

The UCR Bookstore must place a single order of Q dated Homecoming hoodies before the game:

- **Selling Price:** $p = \$50$ per hoodie during Homecoming weekend.
- **Wholesale Cost:** $c = \$30$ per hoodie ordered ($\$20$ **profit margin** on every unit sold).
- **Clearance Salvage Value:** $s = \$10$ per leftover hoodie after Homecoming ($\$20$ **loss** on every unit unsold).
- **Uncertain Demand:** $D \sim N(\mu = 100,\ \sigma^2 = 20^2)$ (rounded/clamped at 0).

The Exact Profit Formula for Any Realized Demand D

$$\text{Units Sold} = \min(D, Q), \qquad \text{Units Leftover} = \max(0, Q - D)$$

$$\boxed{\text{Profit}(Q, D) = 50\min(D, Q) + 10\max(0, Q - D) - 30Q}$$

Why Plugging Average Demand $D = 100$ Into the Profit Formula Lies

Test Order $Q = 100$ on Two Symmetric Days

Suppose we order $Q = 100$ hoodies (cost \$3,000):

- **If Demand is Exactly Average ($D = 100$):**

$$\text{Profit}(100, 100) = 50(100) - 3{,}000 = \mathbf{\$2,000}$$

- **Low Day ($D = 80$, i.e., $\mu - 1\sigma$):** Sell 80, salvage 20:

$$\text{Profit}(100, 80) = 50(80) + 10(20) - 3{,}000 = \mathbf{\$1,200}$$

- **High Day ($D = 120$, i.e., $\mu + 1\sigma$):** Capped at $Q = 100$!

$$\text{Profit}(100, 120) = 50(100) + 0 - 3{,}000 = \mathbf{\$2,000}$$

The Asymmetric Kink Penalty

Average the Low Day ($D = 80$) and High Day ($D = 120$):

$$\frac{\$1{,}200 + \$2{,}000}{2} = \mathbf{\$1,600} \quad \text{(NOT \$2,000!)}$$

- When $D = 80$, you **lose** \$800 vs. the \$2,000 ceiling.
- When $D = 120$, you **gain** \$0 above the \$2,000 ceiling because you ran out of stock!
- **Question:** Should we order $Q = 120$ instead to capture those high-demand days? Let's ask Gemini to simulate 10,000 seasons!

Interactive Exercise 5: Simulate Hoodie Inventory in Gemini

COPY & PASTE THIS PROMPT INTO GOOGLE GEMINI:

```
Run a Monte Carlo simulation of 10,000 trials (using Python in the background with seed 42) for a retail inventory problem. Demand D follows a Normal distribution with mean 100 and standard deviation 20 (clamp negative draws at 0). Selling price p = $50, wholesale cost c = $30, salvage value s = $10. For any order quantity Q, Units Sold = min(D, Q), Leftover = max(0, Q - D), and Profit = 50*min(D,Q) + 10*max(0,Q-D) - 30*Q. Compare Policy A (Q = 100) and Policy B (Q = 120) on the SAME 10,000 random demands. Show: (1) the first 5 trials side-by-side, and (2) a summary table with Mean Profit, Profit Std Dev, Stockout Probability P(D > Q), and Probability Profit < $1,000.
```

Your Tasks (4 Minutes)

1. **By Hand Check:** On a day where demand is $D = 85$ hoodies, calculate exact profit under **Policy A ($Q = 100$)** and **Policy B ($Q = 120$)**.
2. **In Gemini:** Run the prompt above. Which order quantity ($Q = 100$ or $Q = 120$) achieves higher expected profit, and why?

FORMULA & PARAMETER REFERENCE: $\text{Profit}(Q, D) = 50\min(D, Q) + 10\max(0, Q - D) - 30Q$,
$D \sim N(100, 20^2)$, Unit profit = \$20, Unit leftover loss = \$20.

AI Literacy: Auditing Direct LLM Simulations (Trust, But Verify!)

Trap 1: "Hallucinated Prose Numbers"

- If an LLM just writes text without running code, it *guesses plausible-sounding numbers* token-by-token (human/LLM number bias).
- **Fix:** Always include **"using Python in the background"** in your prompt so Gemini executes a real NumPy script!

Trap 2: Silent Formula Misinterpretation

- Even when running Python, an LLM might forget to subtract cost $30Q$ or confuse rate λ with mean $1/\lambda$.
- **Fix (The 2-Row Audit Rule):** Always ask Gemini to print the **first 5 trials** and verify 2 rows with your own calculator!

1. Mandate Background Python Execution → **2. Print First 5 Trials (D_i, Sold, Leftover, π_i)** → **3. Hand-Check 2 Rows Before Trusting $\bar{\pi}$!**

Interactive Exercise 6: Spot the 3 Flaws in This Prompt!

A Manager's Vague Gemini Prompt

```
"Hey Gemini, simulate my bakery's daily croissant profit. Average demand is 200 croissants a day. They sell for $4 each and cost $1.50 to bake. Tell me how much profit I'll make if I bake 220 croissants."
```

Your Diagnostic Task (3 Minutes)

If you paste this prompt into Gemini, Gemini will be forced to **guess three critical pieces of information**. Identify the **3 missing ingredients** and explain how to fix each one!

FORMULA & PARAMETER REFERENCE: Recall the 5-Part Prompt Blueprint: (1) Decision Q, (2) Exact Distribution & Spread σ, (3) Full Payoff/Salvage Rule, (4) Trial Count N & Python Seed, (5) Output Risk Metrics.

Decision Matrix: Direct Gemini Chat vs. Full Python in Colab

Dimension	Mode 1: Direct Simulation in Gemini Chat	Mode 2: Gemini Python Code in Google Colab
Setup Speed	**30 seconds** (paste prompt, read table)	**2 minutes** (paste code into Colab cell, hit Run)
Ideal Trial Size N	$1,000$ to $10,000$ trials	$10,000$ to $1,000,000+$ trials
Transparency & Auditability	Medium (must inspect Gemini's hidden code snippet)	**100% Transparent** (you see and edit every line of Python)
Sweeping 20+ Policies (Q)	Clunky in chat text	**Effortless** (1 loop or vectorized grid + smooth chart)
Queueing & CIs	Good for small traces	**Essential** for 10,000-customer queues & exact 95% CIs

Rule of Thumb for BUS 104

Use **Mode 1 (Direct Chat)** for quick sanity checks and 2-policy comparisons; switch to **Mode 2 (Colab Python)** whenever you need to optimize Q^*, simulate waiting lines, or compute formal Confidence Intervals!

Getting Ready for Lecture 3: Running Gemini Code in Google Colab

Zero Installation Needed

1. Go to `colab.research.google.com` (logged into your UCR Google account).
2. Or click **"Export to Colab"** directly at the bottom of any Python code block generated by Gemini!
3. Press `Shift + Enter` (or the ▶ Play button) to execute $100,000$ trials in under 0.2 seconds.

The 3 Standard Libraries Pre-Installed in Colab

```
import numpy as np          # Fast random arrays
import pandas as pd         # Clean summary tables
import matplotlib.pyplot as plt # Plots & CIs
```

In Lecture 3, you won't need to memorize syntax—you will tell Gemini *what business logic to model*, and Colab will execute the code!

Lecture 2 Key Takeaways

Four Core Ideas from Today

1. **PRNGs Are Deterministic Clock Spinners:** Computers generate $U \sim U(0,1)$ via modular clock arithmetic $X_{n+1} = (aX_n + c) \bmod m$.
2. **Random Seeds Enable Fair Policy Comparisons:** Fixing `seed = 42` guarantees reproducibility and lets us test Policy A ($Q = 100$) vs. Policy B ($Q = 120$) on the **exact same sequence of random demands**.
3. **Inverse CDF & Bucket Lookup:** Any distribution (Uniform, Exponential, Normal, or Empirical) can be simulated by transforming $U \sim U(0,1)$.
4. **Expected Value Hides Tail Risk:** In the 3-Round Gamble, $\mathbb{E}[W_3] = \$8.00$ masked a 50% bankruptcy probability; in Hoodie Inventory, simulation revealed why $Q = 100$ beats $Q = 120$.

Bridge to Lecture 3: Scaling Up with Python, Queueing & CIs

What We Can Already Do

- Choose the right input distribution (U, Exp, N).
- Prompt Gemini Chat to simulate a fixed policy and audit the first 5 trials by hand.

Three Big Questions for Lecture 3

1. **Dual Uncertainty:** What if *both* market price P and demand D are random, and we want to search across all $Q \in [70, 120]$ to find Q^*?
2. **Service Operations:** How do we simulate a dynamic waiting line ($M/M/1$ queue) where each customer's wait depends on the previous customer's departure?
3. **Statistical Precision:** How accurate is a simulation average $\bar{X}$ after n trials? (**95% Confidence Intervals** $\bar{X} \pm 1.96\frac{s}{\sqrt{n}}$).

Lecture 3: Gemini Python Code in Colab, Queueing & CIs

Part 1: Vectorized Python via Gemini (Slides 47–51)

- The 4-Block Blueprint for Python simulation code.
- **Case 3 (Dual-Uncertainty Newsvendor):** Random Price $P \sim U(9, 11)$ & Demand $D \sim N(100, 10^2)$.
- Finding optimal Q^* across a grid of order quantities.

Part 2: Queueing Simulation (Slides 52–56)

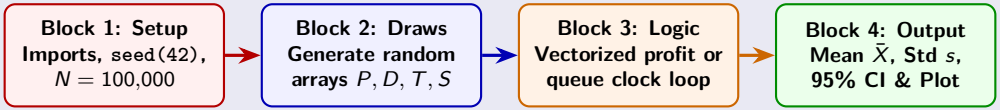
- **Case 4 ($M/M/1$ Service Queue):** Exponential arrivals λ & service μ.
- The 4 Clock Equations for waiting lines.
- Discovering the nonlinear **Utilization Hockey Stick**.

Part 3: Confidence Intervals (Slides 57–65)

- Why $\bar{X}$ varies across seeds.
- Standard Error SE $= \frac{s}{\sqrt{n}}$ vs. single-day risk s.
- **95% CI Formula:** $\bar{X} \pm 1.96 \frac{s}{\sqrt{n}}$.
- Sample size law $n^* = \left(\frac{1.96s}{E}\right)^2$ & comparing policies.

The 4-Block Blueprint of Every Python Simulation Script

When Gemini Writes Python Simulation Code for Colab, Check for These 4 Blocks:



Why Vectorized NumPy Is So Fast

Instead of a slow `for`-loop over $100,000$ independent days, NumPy computes all $100,000$ profits in **one single array line** (`np.minimum(D, Q) * P - c * Q`).

When Do We Still Need a `for`-Loop?

Only when Day/Customer i **depends on the state left behind by** $i-1$—such as a waiting line (Customer i cannot start service until Customer $i-1$ departs!).

Gemini Code Case 3: Dual-Uncertainty Newsvendor

Business Scenario: Wholesale Produce Distributor (Organic Avocados)

Each morning you order Q crates of organic avocados at wholesale cost $c = \$7$ **per crate** with zero salvage value ($s = \$0$). Unlike a textbook problem with a fixed price, you face **two simultaneous sources of uncertainty**:

1. **Uncertain Market Clearing Price:** $P \sim U(9, 11)$ dollars per crate ($\mathbb{E}[P] = \$10$).
2. **Uncertain Daily Demand:** $D \sim N(\mu = 100,\ \sigma^2 = 10^2)$ crates.

Daily Profit Function $\pi(Q, P, D)$

$$\text{Units Sold} = \min(D, Q)$$

$$\boxed{\pi(Q, P, D) = P \cdot \min(D, Q) - 7Q}$$

Managerial Question

Average demand is 100 crates. Should you order $Q = 100$, or does the asymmetric cost ($c = \$7$ loss on unsold crates vs. $\approx \$3$ profit on sold crates) push Q^* below 100?

Reading Gemini's Vectorized Python Code (100,000 Days)

Complete Google Colab Script Generated by Gemini

```
import numpy as np, pandas as pd
np.random.seed(42)
n_trials, c = 100_000, 7.0
# Block 2: Generate 100,000 Random Prices & Demands at Once
P = np.random.uniform(9.0, 11.0, size=n_trials)
D = np.maximum(0, np.random.normal(100.0, 10.0, size=n_trials))
# Block 3 & 4: Evaluate Candidate Order Quantities Q
results = []
for Q in [85, 90, 95, 100, 105, 110]:
    profits = P * np.minimum(D, Q) - c * Q
    results.append({"Q": Q, "Mean_Profit": np.mean(profits),
                    "Std_Dev": np.std(profits, ddof=1),
                    "Stockout_%": 100*np.mean(D > Q)})
print(pd.DataFrame(results).to_string(index=False))
```

Notice how `P` and `D` are generated **outside** the loop so every candidate Q is tested on the **exact same 100,000 market days**!

Interactive Exercise 7: Find Optimal Q^* Under Dual Uncertainty

COPY & PASTE THIS PROMPT INTO GEMINI (OR RUN CODE IN COLAB):

```
Write and run a Python Monte Carlo simulation (seed 42, n = 100,000 trials) for a distributor ordering Q crates at cost c = $7 with salvage s = $0. Market price P is Uniform(9, 11) and demand D is Normal(mean=100, std=10). Daily profit is P * min(D, Q) - 7*Q. Test every integer Q from 80 to 115 on the same 100,000 random days. Print the summary table for Q in [85, 90, 95, 100, 105, 110] and report the exact integer Q* that maximizes expected profit.
```

Your Tasks (4 Minutes)

1. Run the prompt in Gemini (or execute the script from Slide 49 in Google Colab).
2. Which order quantity $Q \in \{85, 90, 95, 100, 105, 110\}$ maximizes expected daily profit $\mathbb{E}[\pi]$?
3. Why is the optimal order quantity Q^* **strictly less than** average demand ($\mu = 100$)?

FORMULA & PARAMETER REFERENCE: $\pi(Q, P, D) = P \min(D, Q) - 7Q$, $P \sim U(9, 11)$ ($\mathbb{E}[P] = \$10$), $D \sim N(100, 10^2)$, Unit profit $\approx \$10 - \$7 = \$3$, Unit leftover loss $= \$7$.

Iterative Modelling: Changing Business Rules in 1 Follow-Up Prompt

Why Simulation Beats Closed-Form Formulas When Reality Gets Messy

In textbook math, adding a new business rule requires re-deriving calculus integrals from scratch. In Colab with Gemini, you simply send a **1-sentence follow-up prompt** to modify Block 3:

1. Goodwill Penalty

"Modify the code so every unmet unit of demand $\max(0, D - Q)$ *incurs a* \$4 *lost-goodwill penalty."*

$\Longrightarrow$ Pushes Q^* up from 95 to 98!

2. Supplier Discount

"Modify the code so wholesale cost drops from \$7.00 *to* \$6.50 *per crate whenever* $Q \geq 100$."

$\Longrightarrow$ Creates a jump at $Q = 100$!

3. 5% Value-at-Risk

"Also report the 5th percentile of daily profit (`np.percentile(profits, 5)`) *for each* Q."

$\Longrightarrow$ Measures worst-case tail loss!

Gemini Code Case 4: Service Operations & Queueing ($M/M/1$)

Business Scenario: Single-Bay Express Oil-Change / Drive-Thru Queue

Cars arrive randomly at a single service bay and are served First-Come, First-Served (FCFS):

- **Random Inter-Arrival Gaps:** $T_i \sim \text{Exp}(\lambda)$, where λ is the arrival rate (e.g., $\lambda = 4$ cars/hr $\implies$ mean gap $1/\lambda = 15$ min).
- **Random Service Durations:** $S_i \sim \text{Exp}(\mu)$, where μ is the service rate (e.g., $\mu = 5$ cars/hr $\implies$ mean service $1/\mu = 12$ min).

Server Utilization Ratio (ρ)

$$\rho = \frac{\lambda}{\mu} = \frac{\text{Arrival Rate}}{\text{Service Capacity}}$$

Here, $\rho = \frac{4}{5} = \mathbf{80\%}$ (the bay is busy 80% of the day).

The Queueing Puzzle

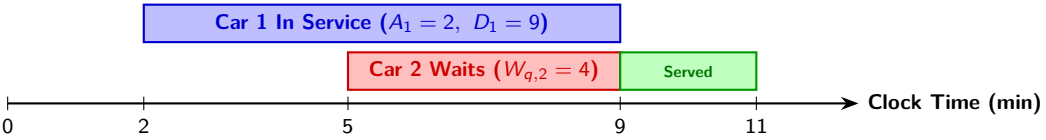
If the service bay has 20% spare capacity ($\rho = 80\% < 100\%$), why do customers still experience long waiting lines? Because **random arrivals clump together while service times fluctuate**!

The 4 Clock Equations Inside Every Queueing Simulator

How a Computer Tracks Customer $i = 1, 2, \ldots, N$ Through a Single Server

Given inter-arrival gap T_i and service duration S_i for Customer i (with $A_0 = 0, D_0 = 0$):

1. **1. Arrival Clock Time:** $A_i = A_{i-1} + T_i$
2. **2. Service Start Time:** $\text{Start}_i = \max\left(A_i,\ D_{i-1}\right)$ **(Must wait if $A_i < D_{i-1}$!)**
3. **3. Wait Time in Queue:** $W_{q,i} = \text{Start}_i - A_i$
4. **4. Departure Clock Time:** $D_i = \text{Start}_i + S_i$



Gemini's Python Code for a 10,000-Customer Queue

Discrete-Event Queueing Script in Google Colab

```
import numpy as np
def simulate_mm1_queue(lam_per_min, mu_per_min, n_cust=10000, seed=42):
    np.random.seed(seed)
    # CRITICAL: NumPy exponential 'scale' is MEAN time (1/rate)!
    inter_arrivals = np.random.exponential(scale=1.0/lam_per_min, size=n_cust)
    service_times  = np.random.exponential(scale=1.0/mu_per_min,  size=n_cust)
    arrivals = np.cumsum(inter_arrivals)
    wait_times = np.zeros(n_cust)
    prev_departure = 0.0
    for i in range(n_cust):
        start_time = max(arrivals[i], prev_departure)
        wait_times[i] = start_time - arrivals[i]
        prev_departure = start_time + service_times[i]
    return np.mean(wait_times), np.mean(wait_times > 15.0)
```

Notice Lines 5–6: We pass `scale = 1.0 / lam_per_min` (the mean wait in minutes), and then loop through the 4 clock equations!

Interactive Exercise 8: Trace a Queue & Find the Hockey Stick

COPY & PASTE THIS PROMPT INTO GEMINI (OR RUN IN COLAB):

```
Write and run a Python discrete-event simulation (seed 42, N = 50,000 customers) for a
single-server M/M/1 queue where mean service time is 1/mu = 6 minutes (service rate mu = 10/hr).
Run the simulation across utilization levels rho = lambda/mu in [0.50, 0.70, 0.80, 0.90, 0.95] by
setting mean inter-arrival time 1/lambda = 6 / rho minutes.  Print a table showing Utilization
rho, Simulated Mean Wait Time W_q (in minutes), Theoretical Mean Wait (rho/(1-rho))*6, and
Probability of Waiting > 15 Minutes.
```

Your Tasks (4 Minutes)

1. **By Hand (2-Car Trace):** Car 1 arrives at $A_1 = 2$ min and needs $S_1 = 7$ min of service ($D_1 = 9$). Car 2 arrives $T_2 = 3$ min later ($A_2 = 5$) and needs $S_2 = 6$ min of service. Find Car 2's Start_2, wait time $W_{q,2}$, and departure D_2.
2. **In Gemini/Colab:** What happens to average customer wait time W_q when server utilization jumps from $\rho = 80\%$ to $\rho = 95\%$?

FORMULA & PARAMETER REFERENCE: $A_i = A_{i-1} + T_i$, $\text{Start}_i = \max(A_i, D_{i-1})$, $W_{q,i} = \text{Start}_i - A_i$, $D_i = \text{Start}_i + S_i$, Utilization $\rho = \lambda/\mu$.

Why Simulation Beats Textbook Queueing Formulas in Practice

Where Textbook Formulas Break Down

The formula $W_q = \frac{\rho}{1-\rho}\frac{1}{\mu}$ requires strict assumptions that rarely hold in real operations:

- Assumes arrival rate λ is **constant 24/7** (ignores morning/lunch rush-hour spikes!).
- Assumes customers wait forever (**zero balking or abandonment**).
- Assumes service times are purely Exponential.

How Simulation Handles Real-World Rules

In Python, adding real-world operations takes 2 lines of code:

- **Rush-Hour Waves:** Let $\lambda(t)$ jump from 4/hr to 12/hr between 11:30 AM and 1:30 PM.
- **Customer Balking:** If line length > 5 cars upon arrival, the customer drives away (lost sale!).
- **Shift Scheduling:** Open a 2nd service bay only during peak hours.

Confidence Intervals Part 1: Why One $\bar{X}$ Is Just an Estimate

The Fundamental Question of Simulation Output Analysis

When you run a simulation of $n = 100$ trials, your sample average $\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$ is **NOT the exact true expected value** $\mu = \mathbb{E}[X]$—it is a random estimate that depends on the seed!

100-Trial Run	Seed 1	Seed 2	Seed 3	Seed 4	Seed 5
Sample Mean Profit $\bar{X}$ ($n = 100$)	$\$1,642.10$	$\$1,718.45$	$\$1,665.80$	$\$1,704.12$	$\$1,673.90$

How Much Should We Trust $\bar{X}$?

If Seed 1 gives $\$1,642$ and Seed 2 gives $\$1,718$, how close is our sample mean $\bar{X}$ to the true μ? We need a **Confidence Interval (CI)** to put an exact margin of error $\pm E$ around $\bar{X}$!

Two Laws Governing Simulation Accuracy: LLN & CLT

1. Law of Large Numbers (LLN)

As the number of simulated trials $n \to \infty$, the sample average converges to the true expected value:

$$\bar{X}_n = \frac{1}{n}\sum_{i=1}^{n} X_i \xrightarrow{n\to\infty} \mu = \mathbb{E}[X]$$

2. Central Limit Theorem (CLT) for $\bar{X}$

Even if single-trial profit X_i is skewed or weird, for large n ($n \geq 30$), the **sample average** $\bar{X}$ follows a tight Normal bell curve centered at μ:

$$\bar{X} \approx N\left(\mu,\ \frac{s^2}{n}\right)$$

Crucial Distinction: s vs. SE $= \frac{s}{\sqrt{n}}$

- **Sample Std. Dev. (s):** Measures **single-day business volatility** (how much one day's profit bounces around). Running more trials n does **NOT** shrink s!
- **Standard Error of the Mean (SE):**

$$\boxed{\text{SE} = \frac{s}{\sqrt{n}}}$$

Measures **uncertainty in our average** $\bar{X}$. Running more trials n shrinks SE toward 0!

The $(1-\alpha)$ Confidence Interval Formula for True Mean μ

Universal Simulation Confidence Interval Formula

Given sample mean $\bar{X}$ and sample standard deviation s from n independent trials:

$$\boxed{\mathrm{CI}_{1-\alpha} = \bar{X} \pm z_{1-\alpha/2} \cdot \frac{s}{\sqrt{n}} = \left[\bar{X} - E,\ \bar{X} + E\right]}$$

where the half-width **Margin of Error** is $E = z_{1-\alpha/2} \cdot \mathrm{SE} = z_{1-\alpha/2} \frac{s}{\sqrt{n}}$.

Confidence	Tail α	Multiplier $z_{1-\alpha/2}$
90% CI	0.10	1.645
95% CI (Std.)	**0.05**	**1.960**
99% CI	0.01	2.576

Plain-English Meaning of a 95% CI

We are 95% confident that the **true mean** $\mu = \mathbb{E}[X]$ lies in $\left[\bar{X} - 1.96\frac{s}{\sqrt{n}},\ \bar{X} + 1.96\frac{s}{\sqrt{n}}\right]$.

Worked Example: Watching the 95% CI Shrink as n Grows

Scenario: Portfolio Simulation with $\bar{X} = \$500$ and Single-Day Volatility $s = \$100$

Watch how the Standard Error $\text{SE} = \frac{100}{\sqrt{n}}$ and 95% Margin of Error $E = 1.96\,\text{SE}$ tighten as we increase n:

Trials n	$\sqrt{n}$	**Std. Error SE $= \frac{100}{\sqrt{n}}$**	**95% Margin $E = 1.96\,\text{SE}$**	**95% Confidence Interval**
$n = 100$	10	\$10.00	$\pm$\$19.60	[\$480.40, \$519.60]
$n = 400$	20	\$5.00	$\pm$\$9.80	[\$490.20, \$509.80]
$n = 2{,}500$	50	\$2.00	$\pm$\$3.92	[\$496.08, \$503.92]
$\mathbf{n = 10{,}000}$	**100**	**\$1.00**	**$\pm$\$1.96**	**[\$498.04, \$501.96]**

Notice What Changed—and What Did NOT Change!

Single-day risk stayed at $s = \$100$, but increasing n from 100 to $10,000$ ($100\times$ more trials) cut the margin of error on the mean by $\sqrt{100} = \mathbf{10}\times$ (from $\pm$\$19.60 down to $\pm$\$1.96)!

The $\sqrt{n}$ Sample-Size Law: How Many Trials n^* Do We Need?

Solving for Required Sample Size n^*

Suppose your boss wants the 95% Margin of Error to be no wider than $\pm E$ dollars:

$$1.96\frac{s}{\sqrt{n}} \leq E \implies \sqrt{n} \geq \frac{1.96\,s}{E}$$

Squaring both sides gives the **Sample-Size Formula**:

$$\boxed{n^* = \left(\frac{1.96 \cdot s}{E}\right)^2} \quad \text{(always round UP!)}$$

The Square-Root Penalty (4× Rule)

Because E sits in the denominator and is **squared**:

- To cut your margin of error E in **half (**$2\times$ **tighter)**, you must run $2^2 = 4\times$ **as many trials**!
- To make your estimate $10\times$ **more precise**, you must run $10^2 = 100\times$ **as many trials**!

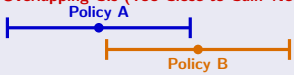
Quick Example ($s = \$100$, Target Error $E = \pm\$2.00$)

$$n^* = \left(\frac{1.96 \times 100}{2.00}\right)^2 = (98)^2 = \mathbf{9,604} \text{ trials}$$

Comparing Business Policies with 95% Confidence Intervals

Visual Decision Rule: Do the CIs Overlap?

Case A: Overlapping CIs (Too Close to Call! Need Larger n)



Case B: Non-Overlapping CIs (Policy B Wins Statistically!)



4 Lines of Python to Print Any 95% CI

```
import numpy as np
n, mean = len(profits), np.mean(profits)
se = np.std(profits, ddof=1) / np.sqrt(n)
ci_low, ci_high = mean - 1.96*se, mean + 1.96*se
print(f"95% CI: [{ci_low:.2f}, {ci_high:.2f}]")
```

Interactive Exercise 9: 95% Confidence Intervals & Sample Size n^*

Scenario: Evaluating a New Dynamic Pricing Algorithm

An e-commerce analytics team simulates a new AI dynamic pricing algorithm for $n = 400$ **simulated days** and observes:

- Sample Mean Daily Revenue: $\bar{X} = \$\mathbf{1,250.00}$; Sample Standard Deviation: $s = \$\mathbf{200.00}$.
- The company's current baseline pricing rule earns a known benchmark of $\$\mathbf{1,220.00}$ per day.

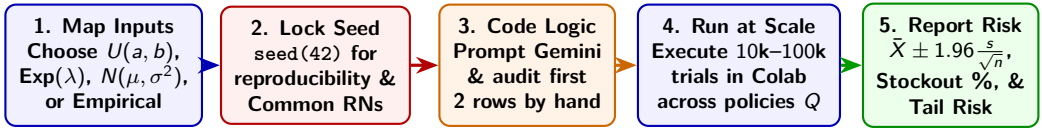
Your Tasks (4 Minutes)

1. **Part A (95% CI & Policy Decision):** Compute the Standard Error SE, the 95% Margin of Error E, and the **95% Confidence Interval** for the new algorithm's true expected revenue μ. Does the new algorithm beat the $\$1,220$ baseline at the 95% confidence level?
2. **Part B (Sample Size Planning):** How many simulated days n^* must the team run to shrink the 95% Margin of Error to $E = \pm\$5.00$?

FORMULA & PARAMETER REFERENCE: $\text{SE} = \dfrac{s}{\sqrt{n}}$, $E = 1.96 \cdot \text{SE}$, $95\%\ \text{CI} = \left[\bar{X} - E,\ \bar{X} + E\right]$,

$n^* = \left(\dfrac{1.96 \cdot s}{E}\right)^2$.

The Complete 5-Step BUS 104 Monte Carlo Simulation Playbook



Checklist Before Presenting Any Simulation to Management

- **Did you avoid the Flaw of Averages?** Never plug $\mathbb{E}[D]$ into a capped/kinked formula—simulate D_i first, then average π_i!
- **Did you check units on Exponential draws?** Remember $\mathbb{E}[X] = 1/\lambda$ (`scale = 1/lam` in NumPy).
- **Did you report both Mean $\bar{X}$ (with 95% CI) AND Tail Risk?** Decision-makers need both expected profit and probability of stockout/ruin.

Three-Lecture Synthesis: From Probability to AI-Powered Simulation

Lecture 1: Distributions

- **Uniform $U(a, b)$:** Flat intervals, $\frac{d-c}{b-a}$, and $a + (b - a)U$.
- **Exponential Exp(λ):** Waiting times, $e^{-\lambda x}$ survival, memorylessness.
- **Normal $N(\mu, \sigma^2)$:** 68–95–99.7 rule, Z-scores, safety stock $\mu + Z\sigma$.

Lecture 2: PRNGs & Gemini

- **Clock Spinner PRNG:** Deterministic $X_{n+1} = (aX_n + c) \bmod m$.
- **Random Seeds:** Reproducibility & fair policy testing.
- **Direct Gemini Simulation:** 3-Round Gamble (50% ruin) & Hoodie Newsvendor.

Lecture 3: Python, Queues & CIs

- **Vectorized Colab Code:** Optimizing Q^* under dual uncertainty.
- **$M/M/1$ Queueing:** 4 clock equations & the $\rho \to 95\%$ hockey stick.
- **95% CIs:** $\bar{X} \pm 1.96\frac{s}{\sqrt{n}}$ and sample size $n^* = \left(\frac{1.96s}{E}\right)^2$.

You are now equipped to build, audit, and scale Monte Carlo simulations using Gemini and Python!